SecureDocs

React JS Study Guide

Project mein use hue saare React concepts —
kya padho, kaise padho, kab tak padho

MUST STUDY | IMPORTANT | GOOD TO KNOW | ADVANCED

Upar wale color codes pure PDF mein har chapter ke badge pe dikhenge — iske hisaab se priority decide karo.

#	Chapter	Priority	Est. Time
1	JSX — JavaScript XML	MUST STUDY	2 days
2	Components & Props	MUST STUDY	3 days
3	useState Hook	MUST STUDY	3 days
4	useRef Hook	IMPORTANT	1 day
5	React Router & useNavigate	MUST STUDY	2 days
6	Event Handlers	MUST STUDY	2 days
7	Conditional Rendering	MUST STUDY	2 days
8	List Rendering — .map()	MUST STUDY	2 days
9	Array Methods	IMPORTANT	3 days
10	Form Handling & Validation	MUST STUDY	3 days
11	setInterval & Async Patterns	IMPORTANT	2 days
12	ES6+ Modern JavaScript	IMPORTANT	4 days
13	CSS — Variables & Dynamic Classes	GOOD TO KNOW	2 days
14	Study Roadmap	—	—

Chapter 1

MUST STUDY

JSX — JavaScript XML

React mein HTML likhne ka tarika — har component yahi use karta hai

JSX kya hai: JavaScript ke andar HTML jaisi syntax likhne deta hai. Browser directly JSX nahi samajhta — Babel tool ise React.createElement() calls mein convert karta hai before running in browser.

JSX ke Important Rules:

Rule	Explanation
class → className	HTML attribute 'class' JSX mein 'className' hota hai — React reserved keyword hai
Single root element	Ek component sirf ek root element return kar sakta hai. Multiple elements ke liye <> (Fragment) use karo
JS expressions {}	Curly braces mein koi bhi JavaScript expression likh sakte ho: {file.name}, {count + 1}
Self-closing tags	Sab tags close hone chahiye: input tag, img tag, br tag (self-closing)
camelCase events	onClick, onChange, onSubmit — HTML wale onclick, onchange nahi chalte
for → htmlFor	Label tag mein for ki jagah htmlFor use hota hai
style as object	Inline style string nahi, object hoti hai: style={{color: 'red', fontSize: 14}}

Project mein JSX kahan use hua:

```
// LandingPage.jsx — Hero section JSX
const HeroLeft = () => (
  <div className="hero-left">        {/* class → className */}
    <h1 className="hero-heading">
      <div className="heading-row1">
        Secure
        <ShieldIcon />               {/* Component as JSX tag */}
      </div>
      <div className="heading-row2">&amp;amp; Leak Guard</div>
    </h1>
    <div className="tagline">
      <div className="tagline-line" />    {/* Self-closing */}
      <span>{taglineText}</span>          {/* JS expression */}
    </div>
  </div>
);
```

Kya padho JSX ke liye:

Resource	Link / Task

Official Docs	react.dev/learn/writing-markup-with-jsx
YouTube	'JSX in React' — Traversy Media ya Codevolution channel
Practice	Ek simple card component banao apne naam aur photo ke saath using JSX

Chapter 2

MUST STUDY

Components & Props

UI ke building blocks — reusable pieces aur unke beech data pass karna

Component ek JavaScript function hai jo JSX return karta hai. Props parent se child ko data pass karne ka tarika hain — ek taraf traffic — sirf parent → child.

Component ke 2 Types (hum Functional use karte hain):

```
// ■ Functional Component — Arrow Function style (project mein yahi use hua)
const Sidebar = ({ active, setActive, collapsed }) => {
  return (
    <aside className="sidebar">
      <nav>...</nav>
    </aside>
  );
};
// Props receive karne ke 2 tarike:
// 1. Destructuring (clean) — hum yahi use karte hain
const TopBar = ({ activePage, alertCount }) => { ... };
// 2. props object (verbose)
const TopBar = (props) => {
  const activePage = props.activePage;
};
```

Props ka flow — SecureDocs project mein:

Prop Name	Syntax	Kaam
active	`active={active}`	Sidebar ko batao kaunsa page selected hai
setActive	`setActive={setActive}`	Sidebar se page change karne ka function
files	`files={files}`	Upload hua files array — Dashboard aur Files page mein share
setFiles	`setFiles={setFiles}`	Files array update karne ka function
downloadHistory	`downloadHistory={history}`	Employee download log — Available Files se History tak
setDownloadHistory	`setDownloadHistory={setFn}`	History update karne ka function

file	`file={fileObject}`	FileDownloadRow ko ek file ka data
onDownloaded	`onDownloaded={callback}`	Download complete hone pe history update
setActive	`setActive={setActive}`	'View all' click pe Files section mein jaao

Kya padho Components & Props ke liye:

Resource	Task
Official Docs	react.dev/learn/your-first-component
Practice Task 1	Ek Navbar component banao jisme logo aur 3 links props se aayein
Practice Task 2	Parent component banao jo child ko name aur age pass kare, child display kare

Chapter 3

MUST STUDY

useState Hook — Component Ki Memory

Component ko data yaad rakhne aur UI update karne ki power deta hai

useState React ka sabse important hook hai. Jab bhi state change hoti hai, component automatically re-render hota hai aur updated UI dikhta hai. Bina useState ke component 'yaad' nahi rakh sakta kuch bhi.

Basic Syntax:

```
const [value, setValue] = useState(initialValue);
//     ↑ current  ↑ update fn    ↑ starting value
//     value                     setValue se update hota hai
```

Project mein use hue saare useState:

State Variable	Kaam
active page	useState('dashboard') — kaunsa sidebar item selected hai
collapsed	useState(false) — sidebar collapse/expand state
files array	useState([]) — admin ke uploaded files ki list
form data	useState({email:'',password:'',role:''}) — login form
errors object	useState({}) — form validation errors
progress bar	useState(0) — upload/download progress 0-100
upload step	useState('idle') — idle/file-selected/analyzing/result
showProfile	useState(false) — topbar dropdown show/hide
showPass	useState(false) — password field eye toggle
downloadHistory	useState([]) — employee download records
search string	useState('') — files page search input
filter type	useState('all') — all/pdf/docx filter button
dragOver	useState(false) — drag zone hover highlight
passForm	useState({current:'',newPass:'',confirm:''}) — password change
editMode	useState(false) — profile edit toggle
result	useState(null) — leak analysis result object

State Update ke 3 Patterns:

```
// Pattern 1: Direct value set karna (simple values ke liye)
setActive("files");
setCollapsed(true);
```

```
setSearch("project");
setStep("analyzing");
// Pattern 2: Previous state use karna (arrays/objects ke liye) — IMPORTANT
// Prev state ka reference milta hai — kabhi direct state mutate mat karo
setFiles(prev => [...prev, newFile]);           // array mein add
setFiles(prev => prev.filter(f => f.id !== id)); // array se remove
setDownloadHistory(prev => [record, ...prev]);  // pehle add karna
// Pattern 3: Object spread — kuch fields update, baaki same
setForm({ ...form, email: e.target.value });
setErrors({ ...errors, email: "" });
setPassForm({ ...passForm, current: e.target.value });
```

Common Mistakes aur Unke Solutions:

Mistake	Solution
State directly mutate karna	files.push(f) — GALAT. Hamesha setFiles([...files, f]) use karo
Async state read karna	setState ke turant baad state read karne se purani value milti hai — functional update use karo
Object state partially update	{role: 'Admin'} — GALAT, baaki fields gum. Hamesha {...form, role: 'Admin'} likho
Re-render loop	setState ko render ke andar directly mat call karo bina condition ke

Kya padho useState ke liye:

Resource	Task
Official Docs	react.dev/reference/react/useState
Practice Task 1	Counter app — increment, decrement, reset buttons
Practice Task 2	Simple todo list — add item, delete item, mark done
Practice Task 3	Toggle dark/light mode button

Chapter 4

IMPORTANT

useRef Hook — DOM Element Direct Access

Re-render trigger kiye bina DOM element ko access karo

useRef ek 'escape hatch' hai — jab tum directly DOM element ko touch karna chahte ho. Yeh state jaisa nahi hai — value change hone pe re-render nahi hota. Project mein file input element ko programmatically click karne ke liye use kiya.

```
// Import
import React, { useState, useRef } from "react";
// useRef declare karna
const inputRef = useRef();  // initial value optional hai
// DOM element se attach karna
<input ref={inputRef} type="file" style={{ display: "none" }} />
// .current se actual DOM element access karna
const handleDropZoneClick = () => {
  inputRef.current.click();  // hidden input ko programmatically click karo
};
// ■■■ Project mein exact use — UploadPage aur LeakDetectionPage ■■■
const UploadPage = () => {
  const inputRef = useRef();
  return (
    <div>
      {/* Hidden file input — user directly nahi dekh sakta */}
      <input
        ref={inputRef}
        type="file"
        accept=".pdf,.docx"
        style={{ display: "none" }}
        onChange={e => { handleFile(e.target.files[0]); e.target.value = ""; }}
      />
      {/* Drop zone click → hidden input trigger */}
      <div className="drop-zone" onClick={() => inputRef.current.click()}>
        Drag & drop or click to browse
      </div>
    </div>
  );
};
```

useRef ke Other Use Cases:

Use Case	Kab use karo
DOM click trigger	Hidden file input ko programmatically click karna — project mein use hua
Focus management	Form submit ke baad first error field pe focus le jaao

Previous value store	Previous render ki value yaad rakhna without re-render
Interval/timeout ID	setInterval ka ID store karna taaki clearInterval kar sako
Scroll position	Element ko scroll karna — scrollIntoView()

Chapter 5

MUST STUDY

React Router v6 & useNavigate

Single Page Application mein pages ke beech navigate karna

React Router ek library hai jo React app mein multiple pages create karne deti hai bina actual page reload ke. URL change hoti hai but page nahi refresh hota. npm install react-router-dom se install hoti hai.

App.jsx Setup — Project ka exact code:

```
// App.jsx — sab routes yahan define hain
import { BrowserRouter, Routes, Route } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>          {/* Poori app ko wrap karo */}
      <Routes>               {/* Sab routes ka container */}
        <Route path="/"                    element={<LandingPage />}       />
        <Route path="/login"               element={<LoginPage />}         />
        <Route path="/signup"              element={<SignupPage />}        />
        <Route path="/admin/dashboard"     element={<AdminDashboard />}    />
        <Route path="/employee/dashboard" element={<EmployeeDashboard />} />
      </Routes>
    </BrowserRouter>
  );
}
```

useNavigate — Programmatic Navigation:

```
// Step 1: Import karo
import { useNavigate } from "react-router-dom";

// Step 2: Component ke andar use karo
const LoginPage = () => {
  const navigate = useNavigate();

  const handleSubmit = (e) => {
    e.preventDefault();
    // Role ke hisaab se redirect
    if (form.role === "Admin") {
      navigate("/admin/dashboard");    // Admin wala page
    } else {
      navigate("/employee/dashboard"); // Employee wala page
    }
  };
  return (
```

```
      <div>
        <button onClick={() => navigate("/signup")}>Sign Up</button>
        <button onClick={() => navigate("/")}>Go Home</button>
      </div>
    );
  };
```

Project mein navigate() calls ka poora list:

navigate() Call	Kab trigger hota hai
navigate('/')	Kisi bhi auth page pe logo click → landing page
navigate('/login')	Sign Up page se 'Sign In' link, Logout button sab jagah
navigate('/signup')	Login page se 'Create account' link
navigate('/admin/dashboard')	Login form — Role=Admin + valid submit
navigate('/employee/dashboard')	Login form — Role=Employee + valid submit

Kya padho React Router ke liye:

Resource	Task
Official Docs	reactrouter.com/docs/en/v6
Practice Task	3 pages banao — Home, About, Contact. Navbar se navigate karo

Chapter 6

MUST STUDY

Event Handlers — User Interactions Handle Karna

Click, type, submit, drag — sab user actions ka response

Event	Project mein use
onClick	Button click, div click — navigate(), setActive(), toggle, delete
onChange	Input/select value change — form fields, search bar, filter
onSubmit	Form submit — login, signup, password change
onDragOver	File drag karne pe — e.preventDefault() + setDragOver(true)
onDragLeave	Drag zone se bahar — setDragOver(false)
onDrop	File drop — e.dataTransfer.files[0] se file nikalo
noValidate	Browser ki default validation disable karo — custom validation use karo

```
// onClick — 3 common patterns
<button onClick={() => navigate("/login")}>Log In</button>  // arrow fn
<button onClick={handleSubmit}>Submit</button>              // function reference
<button onClick={() => setActive("files")}>View all</button>

// onChange — form input
<input
  value={form.email}
  onChange={e => setForm({ ...form, email: e.target.value })}
  onChange={set("email")}  // project mein generic helper use kiya
/>

// onSubmit — e.preventDefault() ZAROORI hai — nahi toh page reload
<form onSubmit={e => { e.preventDefault(); handleLogin(); }}>

// Drag & Drop — Upload page
<div
  onDragOver={e => { e.preventDefault(); setDragOver(true); }}
  onDragLeave={() => setDragOver(false)}
  onDrop={e => {
    e.preventDefault();
    setDragOver(false);
    const file = e.dataTransfer.files[0];
    if (file) handleFile(file);
  }}
>
```

Generic Set Function Pattern — Project mein use hua:

```
// Ek hi function sab form fields ke liye — DRY principle
const set = (field) => (ev) => {
  setForm({ ...form, [field]: ev.target.value });
  setErrors({ ...errors, [field]: "" }); // error clear karo jab type karo
```

```
};
// Use karna:
<input onChange={set("email")} />
<input onChange={set("password")} />
<input onChange={set("role")} />
// Ek hi function — teen jagah kaam aaya
```

Chapter 7

MUST STUDY

Conditional Rendering — Shart ke Mutabiq UI

Kabhi kuch dikhao, kabhi chhupaao — React ke 4 tarike

Method 1 — switch statement (renderContent function):

```
// AdminDashboard.jsx — sidebar se page switching
const renderContent = () => {
  switch (active) {
    case "dashboard": return <DashboardContent files={files} />;
    case "upload":    return <UploadPage files={files} setFiles={setFiles} />;
    case "files":     return <FilesPage files={files} setFiles={setFiles} />;
    case "leak":      return <LeakDetectionPage />;
    default:          return <DashboardContent files={files} />;
  }
};
// Main mein:
<main>{renderContent()}</main>
```

Method 2 — Ternary Operator (?:):

```
// condition ? show_if_true : show_if_false
// Upload page — uploading state
{uploading
  ? <div className="drop-zone-text">Uploading...</div>
  : <>
      <div>Drop file here</div>
      <div>or <span>browse</span></div>
    </>
}
// CSS class dynamically change karna
className={`sidebar ${collapsed ? "sidebar-collapsed" : ""}`}
className={`role-btn ${form.role === r ? "active" : ""}`}
className={`emp-filter-btn ${filter === f ? "emp-filter-active" : ""}`}
// Done state — FileDownloadRow
{state === "done"
  ? <span className="done-badge">Done</span>
  : <button onClick={() => setState("confirm")}>Download</button>
}
```

Method 3 — Short Circuit (&&):

```
// condition && <JSX> — sirf tab render hoga jab condition true ho
{showProfile && <div className="profile-dropdown">...</div>}
{alertCount > 0 && <span className="badge">{alertCount}</span>}
{errors.email && <span className="field-error">{errors.email}</span>}
{success && !uploading && <div className="success-msg">Uploaded!</div>}
{form.password && <div className="strength-bar">...</div>}
{!collapsed && <span className="sidebar-logo-text">SecureDocs</span>}
```

Chapter 8

MUST STUDY

List Rendering — .map() se Dynamic Lists

Array ke har element ke liye JSX banao — har list yahi use karti hai

React mein list render karna matlab .map() use karna. Har element ko unique 'key' prop dena ZAROORI hai — nahi doge toh React warn karega aur performance kharab hogi.

```
// Basic pattern — navItems se sidebar buttons
{navItems.map((item) => (
  <button
    key={item.key}                    // ← KEY ZAROORI HAI — unique hona chahiye
    className={`sidebar-item ${active === item.key ? "active" : ""}`}
    onClick={() => setActive(item.key)}>
    <span>{item.label}</span>
  </button>
))}
// Features grid — LandingPage mein
{features.map((feature, index) => (  // index bhi available hota hai
  <div className="feature-card" key={index}>
    <h3>{feature.title}</h3>
    <p>{feature.desc}</p>
  </div>
))}
// Files table rows — FilesPage mein
{filtered.map(file => (
  <tr key={file.id}>                  // id better key hai index se
    <td>{file.name}</td>
    <td>{fmtSize(file.size)}</td>
    <td>{file.uploadedAt.toLocaleString()}</td>
  </tr>
))}
// Stats cards — AdminDashboard
{cards.map((s) => (
  <div className={`stat-card stat-card-${s.color}`} key={s.label}>
    <div className="stat-card-label">{s.label}</div>
    <div className="stat-card-value">{s.value}</div>
  </div>
))}
```

Key prop ke rules:

Rule	Detail
Unique hona chahiye	Same list ke andar do items ka key same nahi hona chahiye
Stable hona chahiye	Index use karna avoid karo agar list reorder hoti ho — id use karo
String ya number	Key string ya number hona chahiye — object nahi
Sibling mein unique	Different lists mein same key ho sakta hai — no problem

Chapter 9

IMPORTANT

Array Methods — JavaScript Data Manipulation

filter, find, map, reduce aur spread — project mein kadam kadam pe use hue

Method	Syntax	Project Use
.map()	[a,b,c].map(x => x*2)	Har element ke liye naya element banao — list rendering
.filter()	[1,2,3].filter(x => x > 1)	Shart ke mutabiq elements rakho — search, type filter
.find()	[{id:1},{id:2}].find(x=>x.id===2)	Pehla matching element dhundho — page label
.reverse()	[...arr].reverse()	Array ulta karo — latest files pehle dikhao
.slice(n, m)	arr.slice(0, 6)	Array ka hissa lo — sirf 6 recent items
spread [...arr]	[...prev, newItem]	Array mein naya item add karo immutably
filter remove	prev.filter(x=>x.id!==id)	Array se ek item remove karo by id
.includes()	ALLOWED.includes(file.type)	Check karo value array mein hai ya nahi
.length	files.length	Array ki length — count badges
.split()	'Hello World'.split(' ')	String toda array mein — avatar initials
.join()	['H','W'].join('')	Array ko string banao — 'HW'
.toUpperCase()	'pdf'.toUpperCase()	String uppercase — 'PDF'
.toLowerCase()	name.toLowerCase()	Case-insensitive search ke liye

Project ke Real Examples:

```
// 1. Search + Type Filter — AvailableFilesPage
const filtered = assignedFiles.filter(f => {
  const matchSearch = f.name.toLowerCase().includes(search.toLowerCase());
  const matchType   = filter === "all" || filter === f.type;
  return matchSearch && matchType;  // dono conditions true honi chahiye
});
// 2. Latest 6 files dikhao — RecentActivity
{[...files].reverse().slice(0, 6).map(f => <div>{f.name}</div>)}
// [...files] → copy (original change nahi hoti)
// .reverse() → latest pehle
// .slice(0,6) → sirf 6
// 3. File delete karna — FilesPage
setFiles(prev => prev.filter(f => f.id !== id));
// 4. Page label dhundna — TopBar
const pageLabel = navItems.find(n => n.key === activePage)?.label || "Dashboard";
// 5. Avatar initials — "Rahul Sharma" → "RS"
```

```
{name.split(" ").map(n => n[0]).join("")}
// 6. New file add karna — UploadPage
setFiles(prev => [...prev, {
  id: Date.now(),
  name: file.name,
  size: file.size,
  type: file.type,
  uploadedAt: new Date()
}]);
```

Chapter 10

MUST STUDY

Form Handling & Validation

Login, Signup, Password change — controlled inputs aur custom validation

Controlled Component: Input ki value React state mein hoti hai. value={state} aur onChange={setState} dono saath kaam karte hain. Uncontrolled component ka state React ke bahar browser mein hoti hai.

```
// ■■ Controlled Component Pattern ■■■
```

```
  if (Object.keys(e).length) {              // koi error hai?
    setErrors(e); return;                   // errors dikhao, submit rokao
  }
  navigate("/admin/dashboard");             // success → redirect
};
```

Password Strength — Signup Page:

```
const strength = () => {
  const p = form.password;
  if (!p) return 0;
  let s = 0;
  if (p.length >= 8)           s++;  // length
  if (/[A-Z]/.test(p))         s++;  // capital letter
  if (/[0-9]/.test(p))         s++;  // number
  if (/[^A-Za-z0-9]/.test(p))  s++;  // special character
  return s; // 0=empty, 1=Weak, 2=Fair, 3=Good, 4=Strong
};
```

Chapter 11

IMPORTANT

setInterval & Async Patterns

Progress bar animation, timer, aur clearInterval — memory leak avoid karna

```
// setInterval — har N ms pe kuch karo
// Project: Upload progress bar simulate karna
const startUpload = () => {
  setUploading(true);
  setProgress(0);
  const interval = setInterval(() => {      // interval ID save karo
    setProgress(p => {                      // functional update use karo
      if (p >= 100) {
        clearInterval(interval);            // ZAROORI — nahi kiya toh memory leak
        setUploading(false);
        setFiles(prev => [...prev, newFile]);
        return 100;
      }
      return p + 10;                        // har 80ms mein 10% badhao
    });
  }, 80);                                   // 80 milliseconds
};
// Leak Analysis — slower steps ke liye
const iv = setInterval(() => {
  setProgress(p => {
    if (p >= 100) {
      clearInterval(iv);
      setResult(analysisResult);
      setStep("result");
```

```
      return 100;
    }
    return p + 10;
  });
}, 480);  // 480ms — thoda slow = realistic feel
```

setTimeout — ek baar ke liye:

```
// setTimeout — ek baar delay ke baad kuch karo
// Project: Password change success message 3 sec ke baad hide karo
setPassDone(true);
setTimeout(() => setPassDone(false), 3000); // 3 seconds baad false
// Copied badge hide karna
setCopied(true);
setTimeout(() => setCopied(false), 2500);
```

Chapter 12

IMPORTANT

ES6+ Modern JavaScript Concepts

Destructuring, Spread, Arrow functions, Template literals — React ka foundation

Concept	Project Connection
Arrow Functions	() => {} — shorter syntax. Project mein har component aur function
Destructuring	const [a, b] = arr; const {x, y} = obj — props receive karna
Spread Operator	[...arr, new] ya {...obj, key: val} — immutable updates
Template Literals	\`Hello ${name}\` — dynamic strings, className generation
Optional Chaining	obj?.property — null/undefined pe error nahi aata
Nullish Coalescing	value ?? 'default' — null/undefined pe default
Computed Property	{[fieldName]: value} — dynamic key — generic set() function mein
Default Parameters	function f(x = 0) — undefined hone pe default value
Object Shorthand	{name, size} instead of {name: name, size: size}
Ternary Operator	condition ? a : b — conditional rendering mein kadam kadam pe
Logical OR (||)	value || 'default' — falsy hone pe fallback
Logical AND (&&)	condition && — short-circuit rendering

Spread Operator — Project ke examples:

```
// Array spread — add karna
setFiles(prev => [...prev, newFile]);
// Array copy (original change nahi hoti)
[...files].reverse().slice(0, 6)
```

```
// Object spread — partial update
setForm({ ...form, email: "new@email.com" });
setErrors({ ...errors, password: "" });
// History mein pehle add karna
setDownloadHistory(prev => [newRecord, ...prev]);
```

Optional Chaining — project mein:

```
// ?.  — value undefined/null ho toh crash nahi, undefined return hota hai
const label = navItems.find(n => n.key === page)?.label || "Dashboard";
// .find() → undefined → ?.label → undefined → || "Dashboard" → "Dashboard"
// Date format
f.uploadedAt?.toLocaleString("en-IN")
```

Chapter 13

GOOD TO KNOW

CSS — Variables & Dynamic Classes

Dark theme, CSS variables aur React mein dynamic className

```
/* CSS Variables — :root mein define karo, kahin bhi use karo */
:root {
  --emp-bg:      #0d1117;    /* background */
  --emp-accent:  #1d9e75;    /* green accent */
  --emp-text:    #e6edf3;    /* text color */
  --emp-muted:   rgba(230,237,243,0.45);
  --emp-border:  rgba(255,255,255,0.07);
}
/* Use karna */
.sidebar { background: var(--emp-bg2); border: 1px solid var(--emp-border); }
/* Dynamic className — React mein */
```

```
// Template literal se dynamic class
className={`sidebar ${collapsed ? "sidebar-collapsed" : ""}`}
className={`stat-card stat-card-${color}`}  // color = "blue" | "red" etc.
className={`emp-file-row ${state === "done" ? "emp-file-row-done" : ""}`}
// Multiple conditions
className={[
  "emp-sidebar-item",
  active === item.key ? "emp-sidebar-item-active" : "",
].join(" ")}
```

Chapter 14

—

Study Roadmap — Kab Kya Padho

4-week structured plan — zero se project ready tak

Week-by-Week Plan:

Week	Topics	Goal	Practice
W1	JSX, Components, Props	Simple components banao — Navbar, Card	3 components + props pass karo
W1	useState — basic patterns	Counter, Toggle, Text input yaad karo	Todo app banao
W2	Event Handlers (onClick, onChange, onSubmit)	Form handle karo with state	Login form with validation
W2	Conditional Rendering (ternary, &&, switch)	Same component — alag states	Show/hide card component
W3	List Rendering (.map() + key)	Dynamic list with add/remove	Notes list app
W3	Array Methods (filter, find, slice)	Search aur filter implement karo	Searchable todo list
W3	React Router v6 + useNavigate	3-4 page SPA banao	Home, About, Login pages
W4	Form Handling — controlled + validation	Full login/signup form	Registration form with errors
W4	useRef + setInterval	File upload simulation	Progress bar component
W4	ES6+ (Spread, Destructuring, Template literals)	Clean code likhna	Refactor Week 1-3 code
Bonus	Full project review	SecureDocs code samjho	Ek naya feature add karo

Daily Study Routine (Suggested):

Time	Activity
Morning (30 min)	Concept padho — official docs ya YouTube
Afternoon (1 hr)	Code karo — practice task banao
Evening (30 min)	Project ka woh part dekho jahan concept use hua
Weekend	Ek chhota project ya feature build karo

Best Learning Resources:

Resource	Kya milega
react.dev	Official React docs — sabse accurate aur updated

javascript.info	ES6+ JavaScript concepts — beginner friendly
Codevolution (YouTube)	React + React Router series — Hindi-friendly pace
Traversy Media	Project-based React tutorials
reactrouter.com	React Router v6 official docs
W3Schools / MDN	Array methods, String methods reference

Yeh guide tumhare SecureDocs project ke concepts cover karta hai. Har chapter padho, practice tasks karo, aur phir project ka woh file kholo jahan woh concept use hua. Code dekhna + khud likhna = best learning.